

A Project Report

On

Fuselage — Ground Control Station (GCS)

Mission Planner UI (Flutter Web)

Deployed frontend: Vercel

Deployed backend + DB: Railway

This report is generated from the current codebase and deployed services only. No conceptual features are included.

Table of Contents

- 1. EXECUTIVE SUMMARY
- 2. BACKGROUND
 - 2.1 Existing System
 - 2.2 Definition of Problem
 - 2.3 Proposed System
- 3. PROJECT OVERVIEW
 - 3.1 Objective of the project
 - 3.2 Stakeholders
 - 3.3 Scope of project
 - 3.4 Feasibility Analysis
 - 3.4.1 Feasibility study
 - 3.4.2 Technical Feasibility
 - 3.4.3 Operational Feasibility
 - 3.4.4 Schedule Feasibility
 - 3.4.5 Economic Feasibility
- 4. OVERALL PROJECT PLANNING
 - 4.1 Development Environment
 - 4.2 Constraints
 - 4.3 Deliverables
 - 4.4 Assumptions and Dependencies
 - 4.5 Risks
 - 4.6 Process Model
 - 4.8 Testing environment and tools
- 5. ITERATION PLANNING
 - 5.1 Schedule
 - 5.2 Risk
- 6. HIGH LEVEL SYSTEM ANALYSIS
 - 6.1 User Characteristics
 - 6.2 Summary of system features/Functional requirements
 - 6.3 NonFunctional Requirements/Supplementary Specifications
 - 6.4 Glossary
 - 6.5 Business Rules
 - 6.6 Use Case
- 8.3 Operation Contracts

8.4 Reports	
8. DESIGN MODEL	
8.5 Sequence Diagrams	
8.6 Class Diagram	
8.7 UI design	
8.8 Theoretical Background	
8.9 Architecture	
8.10 Database Design	
10. TRANSITION	
10.1 System Implementation	
10.2 System Maintenance	
10.2.1 Corrective maintenance	
10.2.2 Adaptive maintenance	
10.2.3 Preventive maintenance	
10.3 User/Operational Manual	
11. ANNEXURE	
11.1 References	
11.2 Annexure I: User Interview Questionnaires	
11.3 CONCLUSION	

1. EXECUTIVE SUMMARY

Fuselage is a Flutter-based Ground Control Station (GCS) user interface implemented as a map-first mission planner shell. The main screen is structured around a persistent map workspace and operational overlays: top status strip, left navigation hub, right telemetry panel, and a bottom mission dock. In the current repository build, telemetry and mission state are simulated/UI-only. The backend includes a Node.js/Express authentication API backed by PostgreSQL with a single `users` table.

2. BACKGROUND

2.1 Existing System

Before this build, core mission-planner workflows were not unified into a single consistent operator shell in this repository. This milestone focuses on a cohesive UI shell and a minimal backend for authentication.

2.2 Definition of Problem

The problem addressed in this build is the need for a stable, map-first mission planner UI with clear placement of status, tools, mission controls, and telemetry. A secondary need is secure, role-ready authentication so the UI can be accessed via user accounts.

2.3 Proposed System

Implemented scope: a Flutter web UI shell (map + overlays) plus an authentication backend (Express + PostgreSQL). Non-implemented scope explicitly excluded from this report: MAVLink transport, real autopilot mission protocol, real calibration flows.

3. PROJECT OVERVIEW

3.1 Objective of the project

To deliver a usable mission planner / GCS UI shell with modular navigation, map overlays, and telemetry presentation, and to provide a production-friendly authentication API for user access.

3.2 Stakeholders

- Pilot/Operator
- Admin
- Supervisor/Operations Officer
- Field Technician

3.3 Scope of project

Included: map rendering (flutter_map), basemap switching, mission geometry toggle, waypoint demo flow, map HUD, flight trail, right telemetry panel (simulated), alert banners (simulated triggers), bottom mission dock controls, tools sheet routes, auth screens. Excluded: live vehicle connection, real command execution, mission upload/download to FC, and persistent mission storage.

3.4 Feasibility Analysis

3.4.1 Feasibility study

The project is feasible within the current toolchain: Flutter for a cross-platform UI, and Node/Express + PostgreSQL for backend services. The repository includes Docker and deployment configuration supporting repeatable builds.

3.4.2 Technical Feasibility

Frontend dependencies are declared in pubspec.yaml (flutter_map, geolocator, camera, shared_preferences, http). Backend dependencies are declared in backend/package.json (express, pg, bcryptjs, jsonwebtoken, dotenv, cors). The API provides /health and /auth endpoints; the database schema is provided as backend/sql/init.sql.

3.4.3 Operational Feasibility

The UI is designed for operators with basic computer familiarity and supports a clear map-first workflow. Critical actions are UI-guarded (e.g., emergency stop long-press) and simulation labels clarify what is not live.

3.4.4 Schedule Feasibility

A detailed schedule is not tracked as a formal artifact in the repository. However, the project is modular and can be advanced incrementally (UI shell, auth backend, deployment configs).

3.4.5 Economic Feasibility

The build uses open-source tooling (Flutter SDK, Node.js, PostgreSQL) and common deployment platforms (Railway and Vercel), reducing licensing costs for the current milestone.

4. OVERALL PROJECT PLANNING

4.1 Development Environment

Operating system used in this workspace: Windows.

Frontend: Flutter Web (Chrome).

Backend: Node.js (Express).

Database: PostgreSQL.

Deployment: Railway (API + Postgres), Vercel (web frontend).

4.2 Constraints

- Simulated telemetry and UI-only command handling in the current milestone.
- No mission persistence to database or autopilot.
- CORS must be configured to the frontend domain for browser access.
- Real vehicle link (MAVLink) not present.

4.3 Deliverables

- Flutter UI shell (Fuselage) deployed on Vercel
- Express auth API deployed on Railway
- PostgreSQL schema: users table (backend/sql/init.sql)
- Documentation: docs/GCS_Report.md and docs/GCS_User_Manual.md

4.4 Assumptions and Dependencies

Assumptions:

- Users can operate a web UI and map interactions.
- Network connectivity is available for hosted frontend/backend.

Dependencies:

- Flutter SDK and dependencies listed in pubspec.yaml
- Node dependencies listed in backend/package.json
- Railway/Vercel build environments and configuration

4.5 Risks

- Vehicle integration risk: adding MAVLink later can surface protocol/timing edge cases.
- Performance risk: overlay-heavy UI and high-frequency telemetry require throttling on web.
- Deployment risk: misconfigured environment variables (AUTH_API_URL, DATABASE_URL, CORS_ORIGIN) can break runtime.

4.6 Process Model

A formal process model (e.g., Waterfall/Agile) is not defined as a document artifact in the repository. Observed practice in the repo is incremental delivery: implement UI shell components, add backend endpoints, and deploy frequently.

4.8 Testing environment and tools

Frontend testing: Flutter Web run in Chrome (flutter run -d chrome).

Backend testing: HTTP calls to /health, /auth/register, /auth/login.

Database testing: SQL run in Railway Postgres console (verification queries on users table).

5. ITERATION PLANNING

5.1 Schedule

A formal iteration schedule (task duration table) is not stored in the repository. The current milestone includes UI shell + auth backend + deployment configuration.

5.2 Risk

Key iteration risks observed:

- Deployment configuration drift between environments.
- UI changes affecting layout responsiveness.
- Backend auth changes requiring coordinated frontend rebuild (AUTH_API_URL baked at build time).

6. HIGH LEVEL SYSTEM ANALYSIS

6.1 User Characteristics

Users are expected to have basic knowledge of web/app usage and map interactions. They should be comfortable with login, navigation panels, and interpreting status indicators.

6.2 Summary of system features/Functional requirements

User Management:

- Register and login via Express API (JWT + bcrypt)

GCS Shell:

- Map-first UI with top status bar, sidebar, right telemetry, bottom dock

Map & Mission (demo):

- Add waypoint hint + long-press add, mission geometry toggle, mission summary gating (NFZ/pre-arm demo)

Telemetry (simulated):

- Telemetry deck values and health labels; alert banner types

6.3 NonFunctional Requirements/Supplementary Specifications

- Responsiveness: width clamping and scroll to avoid overflows on web.
- Usability: consistent tactical styling, map as primary workspace.
- Security: hashed passwords; JWT tokens; restricted CORS origin for deployed web.
- Reliability: /health endpoint checks DB connectivity.
- Maintainability: modular Flutter widgets and centralized routing.

6.4 Glossary

Term	Description
Fuselage	Project name used for the GCS UI shell.
GCS	Ground Control Station interface.
Waypoint	A coordinate point used in the mission route (demo).
Telemetry	Flight metrics shown in the right panel (simulated).
NFZ	No-fly-zone overlay used for demo gating.
JWT	Token format used by the backend for auth sessions.

6.5 Business Rules

- Email must be unique for each user (database UNIQUE constraint).
- Password is stored only as a hash (bcrypt).
- JWT token is issued on successful login/register.
- Mission actions that imply vehicle control are UI-only (no live actuation).

6.6 Use Case

Login: user enters email/password; on success a JWT token is returned and the UI navigates to the shell.

Register: user provides name/email/password; account is inserted to users table and token returned.

Map operations: user views the map, toggles layers, and uses demo waypoint creation and mission summary.

Tools: user opens tools sheet to access feature routes (telemetry, camera, manual control, etc.).

8.3 Operation Contracts

Operation	Inputs	Postconditions (actual behavior)
GET /health	—	Returns {status: ok, db: up} if DB connection works
POST /auth/register	name, email, password, pilotId?	Inserts users row (role=pilot), returns token + user
POST /auth/login	email, password	Validates bcrypt hash, returns token + user payload

8.4 Reports

The repository does not include implemented reporting modules backed by persistent mission/telemetry tables. Current reportable artifacts are:

- API health status (/health)
- Database content for users (users table)
- UI state/telemetry shown in the interface (simulated, not persisted).

8. DESIGN MODEL

8.5 Sequence Diagrams

No sequence-diagram images are stored in the repository. The implemented request/response sequences can be described:

Login (API): Client → POST /auth/login → DB SELECT user by email → bcrypt compare → JWT sign → Client receives token.

Register (API): Client → POST /auth/register → DB check existing → bcrypt hash → DB INSERT user → JWT sign → Client receives token.

8.6 Class Diagram

No UML class-diagram file is present in the repository. The implemented code structure includes these real classes/modules:

- Flutter: MissionPlannerApp, SplashScreen, LoginScreen, RegisterScreen, GcsShell, MapScreen, GcsStatusModel, GcsRightTelemetryPanel, GcsBottomDock
- Backend: server.js (Express app), auth.js (routes), db.js (pg Pool wrapper)

8.7 UI design

Auth:

- Pilot Sign In (login) and Pilot Registration (register)

GCS Shell:

- Top status bar (link, latency, GPS, power, mode/state)
- Left sidebar (Map/Mission/Logs/Settings/Tools)
- Map workspace (basemap + overlays)
- Right telemetry panel (alt/speed/battery/GNSS/health/heading)
- Bottom dock (Start/Load/Save/Pause/RTH/Waypoints/Summary/Track/E-Stop; some placeholders)

8.8 Theoretical Background

Frontend (Flutter): uses widget composition and route-based navigation. The main layout is a stack of overlays around a primary map widget.

Backend (Express): request/response JSON API for authentication and health checks.

Database (PostgreSQL): relational storage for user records with constraints and defaults.

Security: bcrypt password hashing and JWT tokens for authenticated access.

8.9 Architecture

Deployed architecture (actual):

- Vercel hosts the Flutter web build.
- Railway hosts the Express API service.
- Railway Postgres stores the users table.
- The Flutter build is configured with AUTH_API_URL pointing to the Railway API origin.
- The API CORS is configured via CORS_ORIGIN.

8.10 Database Design

The implemented schema in backend/sql/init.sql defines a single table: users.

Fields: id (BIGSERIAL PK), name (TEXT), email (TEXT UNIQUE), password_hash (TEXT), pilot_id (TEXT nullable), role (TEXT default pilot), created_at (TIMESTAMPTZ default now()).

Field	Type	Constraints
id	BIGSERIAL	PRIMARY KEY

name	TEXT	NOT NULL
email	TEXT	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL
pilot_id	TEXT	NULL
role	TEXT	NOT NULL, DEFAULT 'pilot'
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

10. TRANSITION

10.1 System Implementation

Backend deployment: Railway service running Node/Express; database deployed as Railway Postgres.

Database initialization: backend/sql/init.sql executed in Postgres console.

Frontend deployment: Vercel builds Flutter web output directory (build/web) with AUTH_API_URL set.

10.2 System Maintenance

Maintenance tasks observed for this deployed setup include environment variable updates (CORS_ORIGIN, JWT_SECRET, DATABASE_URL), redeploying services after changes, and monitoring /health for DB connectivity.

10.2.1 Corrective maintenance

Fixing runtime failures such as DB connectivity errors and deployment build failures; confirming via /health and logs.

10.2.2 Adaptive maintenance

Updating configuration for new domains, changing build-time AUTH_API_URL for frontend rebuilds, and rotating secrets.

10.2.3 Preventive maintenance

Proactive tightening of CORS_ORIGIN, strong JWT_SECRET, and routine backups/monitoring on hosted services.

10.3 User/Operational Manual

The repository includes a user manual: docs/GCS_User_Manual.md. It describes the layout (top bar, sidebar, map, telemetry panel, bottom dock) and clarifies what is simulated vs real.

11. ANNEXURE

11.1 References

Repository files referenced:

- docs/GCS_Report.md
- docs/GCS_User_Manual.md
- backend/README.md
- backend/sql/init.sql
- backend/src/server.js, backend/src/auth.js, backend/src/db.js
- lib/core/routing/app_router.dart

11.2 Annexure I: User Interview Questionnaires

No user interview questionnaire document is included in the repository at this time.

11.3 CONCLUSION

Fuselage provides a functional, map-first GCS UI shell with simulation-driven telemetry and mission interactions, and a working authentication backend persisting user accounts in PostgreSQL. The deployed system is accessible via Vercel (frontend) and Railway (backend + database). Real vehicle integration features are not part of the current repository milestone and are therefore not included beyond explicit non-implementation statements.